

A02 - Mothership Mayhem

Welcome aboard captain! Your team at UNC Deep Space Systems wants to build a modular mothership capable of swapping power sources on the fly, firing thrusters, and running science experiments. You'll build tiny modules that communicate cleanly with proper encapsulation. You'll use dependency injection to refuel or change your power source mid-mission without touching mothership code!

How the Classes Work Together

<code>AModule.java</code>	# Abstract base class for all modules
<code>ExperimentModule.java</code>	# Runs and reports science experiments
<code>FuelGenerator.java</code>	# Fuel-powered implementation of <code>IPowerGenerator</code>
<code>IModule.java</code>	# Interface that requires the status update method for all modules
<code>IPowerGenerator.java</code>	# Interface for any power generator
<code>Main.java</code>	# The entry point for all java programs.
<code>Mothership.java</code>	# Orchestrates modules (power, thrusterModule, experiments)
<code>SolarGenerator.java</code>	# Solar-powered implementation of <code>IPowerGenerator</code>
<code>ThrusterModule.java</code>	# Consumes power to provide thrust

The **Mothership** coordinates a collection of **AModules**—most notably a **ThrusterModule**, an **ExperimentModule**, and one injected **IPowerGenerator**. The power generator can be either a **SolarGenerator** or a **FuelGenerator**, both of which *implement* the `IPowerGenerator` interface so the mothership can call `generatePower()` without caring how the power is produced. This generator is provided to the mothership via **dependency injection**, which means you can choose which one to use at runtime by handing the mothership a different generator instance—no changes to mothership code required.

The mothership is composed of its `thrusterModule` and `experiment` modules and may maintain a list of all `AModule` objects to broadcast `statusReport(...)` calls for consistent health reporting. The **Main** class acts as the mission script: it wires everything together, injects the chosen power generator, and triggers high-level actions such as `requestPower()`, `fireThruster(...)`, `runExperiment()`, and reporting routines.

Inheritance and Interfaces

- **AModule** is an **abstract** base class. It defines shared behavior (e.g., a standardized `statusReport(...)`) that all concrete modules inherit, but it should not be instantiated by itself—there's no such thing as a generic, fully functional module. Marking it *abstract* prevents accidental construction of incomplete objects and communicates design intent.
- **IPowerGenerator** is an **interface** that specifies *what* a power source must do (e.g., `generatePower()`), without specifying *how*. Concrete classes like **SolarGenerator** and **FuelGenerator** implement this interface with their own strategies.

Dependency Injection (DI) in Practice

The mothership does **not** instantiate its power generator directly. Instead, the chosen generator (e.g., `new SolarGenerator()` or `new FuelGenerator(fuel)`) is **injected** into the mothership, either through the constructor (constructor injection) or through a setter method (setter injection). By allowing the class to take in an interface type, it decouples the mothership from any one power source, enabling a flexible solution with whatever module is available. It also makes it easier for testing (mock generators), and cleaner upgrades without changing mothership code.

Apprentice

This stage is where we lay the **foundation**: base class and associated modules. In this section, you will work with the following files:

- `IModule.java` # Interface that Requires the status update.
- `AModule.java` # Abstract base class for all modules
- `ThrusterModule.java` # Consumes power to provide thrust

To begin, take a look at `IModule`:

```
public interface IModule {
    String getName();
    void statusReport(String moduleStatus, boolean isSuccessful);
}
```

The interface defines the functionality necessary for all of our modules. We can infer that there will be a name instance variable. Your job is to create the infrastructure necessary to create a category of modules that are interchangeable via dependency injection. To implement the `IModule` interface, we will create an abstract parent class to hold all of our information.

Why Mark a Class Abstract?

Use an abstract class when:

1. You want to **share code and a common API** among related subclasses (e.g., one consistent `statusReport(...)`), and
2. It **doesn't make sense** to create a base instance on its own (a generic `AModule` isn't operational).

This keeps your codebase safe (no half-baked base objects), clear (intent is explicit), and DRY (common logic lives in one place).

`AModule` (`AModule.java`)

The stub is there for you, but you must fill in the details. Create an **abstract** class called `AModule` that implements the `IModule` interface. This class will be your base class that all other modules will inherit from.

Fields and Constructors

- A `String` representing the **name** of the module.

The value for the name should be passed to the constructor as a parameter and set in the body.

Methods Remember that abstract classes can contain both abstract and concrete methods. In this case, we don't need any abstract methods, *even though* we have an abstract class. This class will have two **concrete** (non-abstract) methods:

- `String getName()`: a standard getter for the name Variable.
- `statusReport` with the following parameters:
 - `String moduleStatus`
 - `boolean isSuccessful`

Your method should first print the name of the module, and the current status, simulating the start of a particular task (initializing, processing, finished, etc.): Example output: `<moduleName> is Initializing ...` Then it should check to see whether the action was successful by checking the boolean. If the action is successful, it should print: `Action Successful`. Otherwise, it should print the status as the following string: `<moduleName> needs immediate attention!`

`ThrusterModule` (`ThrusterModule.java`)

Create a subclass of `AModule` called `ThrusterModule`.

Fields:

- `int` `fuel` - which will initially have a value of 100
- `boolean` `lastFired` - which will initially be `false`

Note: initialize your fields in your constructor. These have default values and therefore do not need to be taken in as parameters.

Methods:

1. Modify `statusReport` to make it print the amount of fuel remaining in `thrusterModule` as well as whether it last fired. Then, call your base implementation to print the remaining information. Example output: `ThrusterModule: 80 units of fuel remaining. Last fired: Yes`
2. Create a method called `thrust` which takes `int` `availablePower` as a parameter and returns a `boolean`.

```
public boolean thrust(int availablePower);
```

- This method performs a thrust each time it is called.
- For a successful thrust, you need to have at least 5 units of fuel and 5 units of available power.
- If you have enough fuel and power, use 5 units of fuel and set `lastFired` to true.
- Each time you thrust, you need to decrement your fuel value.
- Print the following line `"ALERT Captain: 5 fuel used for propulsion maneuver."`
- Return `true`
- Otherwise, set `lastFired` to false, print `"ThrusterModule: Not enough power or fuel to fire ."` and return `false`.

Enchanter

In this stage of the assignment, you will create your generators! It is important that we specify *what* a power source must do (e.g., `generatePower()`), without specifying *how*. Therefore, we are going to introduce the `IPowerGenerator` interface, while classes like `SolarGenerator` and `FuelGenerator` implement this interface with their own strategies:

```
public interface IPowerGenerator extends IModule{
    int generatePower();
}
```

Notice that `IPowerGenerator` extends the `IModule` interface. That means that all generators have an **IS-A** relationship with `IModule`. The interface extends the functionality to include the requirements from `IModule` and adds on the `generatePower()` method.

Now, you'll create two classes that both implement `IPowerGenerator` and inherit from `AModule`.

SolarGenerator (SolarGenerator.java)

Change the class header to use the `AModule` parent class and the `IPowerGenerator` interface.

Methods: *hint: remember your super keyword. You'll use it more than once*

1. Constructor: `SolarGenerator` extends `AModule`, and you will have to make a constructor to initialize the object.
2. `statusReport`: As is, `SolarGenerator` inherits `statusReport` from `AModule`. However, we need additional information for our generator classes. Keeping in mind the DRY principle, print out the following solar generator specific line: `Solar Generators will never die` Then, print the lines from your base implementation.
3. Implement `generatePower` to return a power of 10.

FuelGenerator (FuelGenerator.java)

Fields and Constructor:

- `int fuel` - initialize as a constructor parameter

Methods:

1. Modify `statusReport` to print the following fuel generator specific line before printing the lines from `AModule`'s implementation. `FuelGenerator: <fuel> units of fuel remaining.`
2. Implement `generatePower` as follows:
 - If `fuel >= 10`, return `10` and decrement fuel by that amount.
 - Otherwise, return the remaining fuel and set fuel to 0.

Simple reference:

Class	<code>generatePower()</code>	Tracks State	<code>statusReport</code> adds...
<code>SolarGenerator</code>	Constant output (e.g., 10)	No	Solar note, then base report
<code>FuelGenerator</code>	Consumes fuel in chunks (10 or remainder)	Yes (fuel)	Fuel remaining, then base report

Sorcerer

Now, for the fun part where everything comes together! We are going to make an experiment module that has a little more complexity.

ExperimentModule (ExperimentModule.java)

Fields and Constructor:

- `String experimentName`
- `double[] parameters`
- `double result`
- `boolean hasRun`

The constructor should take in the `experimentName` and `parameters` as arguments and then set `hasRun` to `false`, and `result` to 0;

Methods:

1. `void runExperiment()` - Iterate through the list of parameters, and increment result by each multiplied by a random number. (use `Math.random()`). Set `hasRun` to `true`.
2. `String getSummary()` - This method should return one of the two following lines depending on the status of `hasRun`:

```
return "Experiment not run yet.";
```



```
return "Experiment '" + experimentName + "' result: " + result;
```
3. `statusReport` - override the existing implementation to print the following: `ExperimentModule: experimentName completed.` OR `ExperimentModule: experimentName pending.` depending on the status of `hasRun`. Then, print the lines from your base implementation.

Mothership (Mothership.java)

Fields:

- Create private instances of `IPowerGenerator`, `ThrusterModule`, `ExperimentModule`, and an `ArrayList` of `IModule`.

Use constructor injection to initialize these fields and add them to your List of `IModules`:

```
public Mothership(IPowerGenerator powerGenerator, ThrusterModule thrusterModule,
    ExperimentModule experimentModule);
```

Thought experiment: Why is `ArrayList` bound to `IModule`? In this case the java generics allow us to hold all the objects that have an **IS-A** relationship with `IModule` which is what we want. We could also choose `AModule`, but that would tightly couple our implementation to existing classes. Interfaces are more flexible.

Methods:

1. `int requestPower()` - this method requests power from your power generator
2. `boolean fireThruster(int availablePower)` - this method uses your `thrusterModule` to thrust forward
3. `void runExperiment()` - delegate to your experiment module
4. `void printStatusReports()` - iterate modules and call `statusReport("Normal", true)`
5. `String getExperimentSummary()` - delegate and return the summary from the `experimentModule`.

—

Mission Control (Main.java)

This is your **launch sequence**: wire the ship together, exercise its core capabilities, and print a few simple readouts so we can verify behavior end-to-end.

What to build

1. **Choose a power source** Create an `IPowerGenerator` instance. Start with a `SolarGenerator`; later, verify hot-swappability by replacing it with a `FuelGenerator` (e.g., initialized with a reasonable fuel amount). This demonstrates **dependency injection** in action.
2. **Assemble the modules** Construct a `ThrusterModule` (it should initialize with its default fuel and flags) and an `ExperimentModule` (give it a short name and a small array of `double` parameters).
3. **Launch the mothership** Use **constructor injection** to pass the generator, thruster, and experiment into the `Mothership`. After construction, ensure the mothership internally registers these in its modules list.
4. **Request power and report it** Call `requestPower()` on the mothership, store the returned value, and print a one-line message confirming the requested power so the output is human-readable.
5. **Fire the thruster** Call `fireThruster(...)` using the power you just requested. Capture the boolean result and print whether the thruster fired. (If the attempt fails, your output should still clearly explain why.)
6. **Run the experiment** Delegate to the `ExperimentModule` through the mothership, then print the experiment's summary line.
7. **Print status reports** Finally, call `printStatusReports()` on the mothership to output the health and activity of all modules.

Merlin: Extra Credit

Note: You are only eligible for extra credit if you have 100% on the rest.

Now, it's time to get creative!

Now that you have demonstrated successful mastery of inheritance, polymorphism, encapsulation, and dependency injection you have the option of earning some extra points. For your implementation to be considered for extra credit it must do the following things.

1. Be passed into `Mothership.java`
2. Inherit from the `AModule` Abstract class
3. Print out something new in its `statusReport`. Feel free to be creative on the theme of your module.
4. Add an overloaded constructor to `mothership` that will take in another module as a parameter, and add it to the list of `ShipModules`.
5. Add it as a call in the main method

Extra credit will be awarded after the deadline through a manual grading process.